

Implementing a calculator: answers

Luca Roversi

December 9, 2025

Learning outcomes

- “Step-wise” definition of a language for numerical expressions
- “Step-wise” construction of an interpreter:
 - for numerical expressions with sums and multiplications of constants
 - valued in **Integer** and **BN**
 - based on applicative functors

“Vanilla” solution?

The very likely one.

- `Arithmetic.Vanilla.Expressions.hs`
- `Arithmetic.Vanilla.Interpreter.hs`

Questions.

- Can we take advantage from applicative functors?
- Can we step-wise define both expressions and corresponding interpreters from simpler up to more complex expressions?

A solution in Haskell based on Functors

Set to collect results. `Arithmetic.Result.hs`

Constants only.

- `Arithmetic.Expressions.Const` (applicative functor)
- `Arithmetic.Interpreters.Functorial.Const.Eval.hs`

Addition.

- `Arithmetic.Expressions.Add` (functor)
- `Arithmetic.Interpreters.Functorial.Add.Eval.hs`

Addition and multiplication.

- `Arithmetic.Expressions.Mul` (functor)
- `Arithmetic.Interpreters.Functorial.Mul.Eval.hs`

Recurrent (incremental) patterns for Evals

```
intToInt :: Terms Int -> Result Int
intToInt (Tcon c) = pure c
intToInt (Tadd x y) =
    (<*>) (fmap (+) (intToInt x)) (intToInt y)
intToInt (Tmul x y) =
    (<*>) (fmap (*) (intToInt x)) (intToInt y)

bnToBN :: Terms BN -> Result BN
bnToBN (Tcon c) = pure c
bnToBN (Tadd x y) =
    (<*>) (fmap (+) (bnToBN x)) (bnToBN y)
bnToBN (Tmul x y) =
    (<*>) (fmap (*) (bnToBN x)) (bnToBN y)
```

- A new interpreting function for every new expression constructor
- Identical patterns for Int/Integer and BN